

Lab Instructions

I'd run this lab as a **guided experiment**, not just a coding exercise: students should first **predict**, then **run**, then **explain**.

Python's `concurrent.futures` provides a high-level interface for running callables asynchronously using either `ThreadPoolExecutor` or `ProcessPoolExecutor`. Its `map()` method can execute calls concurrently, and for `ProcessPoolExecutor`, `chunksize` can significantly affect performance for long iterables. Python's concurrency documentation also emphasizes that tool choice depends on whether the workload is **CPU-bound** or **I/O-bound**. [\[docs.python.org\]](https://docs.python.org/3/library/concurrent.futures.html) [\[docs.python.org\]](https://docs.python.org/3/library/concurrent.futures.html)

Learning Objectives

Students will:

1. Benchmark a **serial baseline**.
 2. Compare **threading** and **multiprocessing**.
 3. Distinguish **CPU-bound** and **I/O-bound** workloads.
 4. Measure:
 - runtime
 - speedup
 - efficiency
 - correctness
 5. Explain why parallel code is sometimes faster, slower, or no better.
-

Lab Structure

Part 0 — Prediction Before Coding

Predict:

1. Which will be fastest for a CPU-heavy pure Python function?
 - Serial

- Threads
 - Processes
 - NumPy
2. Which will be fastest for an I/O-like task using `time.sleep()`?
- Serial
 - Threads
 - Processes
3. What happens if we use too many workers?

Write down the predictions before running code.

Recommended File Structure

For Day 1, I recommend students use a **script** rather than only a notebook, because `ProcessPoolExecutor` is more reliable when run from a `.py` file.

day1_lab/

└─ run_day1_lab.py

└─ results.csv

└─ README.md

Full Example Code: run_July22_lab.py

(download the code from the class website)

<https://cosmos-ai-ucm.github.io/modules/scaling-parallelism>

Suggested Student Workflow

Step 1 — Run the baseline

Students first run:

```
python run_day1_lab.py
```

Record:

CPU count:

Serial CPU time:

Best threaded CPU time:

Best process CPU time:

Serial I/O time:

Best threaded I/O time:

Best process I/O time:

NumPy vectorized time:

Step 2 — Fill in a Results Table

Students should create a table like this in their report:

Experiment	Workers	Mean Runtime	Speedup	Efficiency	Correct?
CPU serial	1		1.00	1.00	yes
CPU threads	2				yes
CPU threads	4				yes
CPU processes	2				yes
CPU processes	4				yes
I/O serial	1		1.00	1.00	yes
I/O threads	4				yes
NumPy vectorized	1				yes

Optional Notebook Version

If you want to run this in Jupyter, I recommend doing **threading and NumPy inside the notebook**, but running the process-based benchmark as a .py script.

In a notebook cell:

```
import subprocess
```

```
import sys
```

```
result = subprocess.run(  
    [sys.executable, "run_day1_lab.py"],  
    capture_output=True,  
    text=True  
)
```

```
print(result.stdout)
```

This avoids many common multiprocessing issues in notebooks.

Additional Experiment 1: Chunk Size

After students run the main lab, modify this line:

```
run_process_cpu(cpu_data, w, chunksize=100)
```

Try:

```
chunksize=1
```

```
chunksize=10
```

```
chunksize=100
```

```
chunksize=1000
```

```
chunksize=5000
```

Add this helper experiment:

```
def experiment_chunksize(cpu_data, workers=4, repeats=3):  
    chunksizes = [1, 10, 100, 1000, 5000]
```

```
rows = []
```

```
serial = time_function(  
    "CPU serial",  
    lambda: run_serial_cpu(cpu_data),  
    repeats=repeats  
)
```

```
for chunksize in chunksizes:
```

```
    row = time_function(  
        f"processes workers={workers}, chunksize={chunksize}",  
        lambda c=chunksize: run_process_cpu(cpu_data, workers, chunksize=c),  
        repeats=repeats  
    )  
    row["workers"] = workers  
    rows.append(row)
```

```
print_table(rows, baseline_time=serial["mean"])
```

Then call it inside main():

```
experiment_chunksize(cpu_data, workers=4, repeats=3)
```

Discussion Questions

1. Why is chunksize=1 often slow?
 2. Why can a huge chunksize hurt load balance?
 3. What chunksize worked best on your machine?
 4. Would the best chunksize be the same for images, text, and CSV data?
-

Additional Experiment 2: Larger vs Smaller Task

Have students change:

```
for i in range(8_000):
```

to:

```
for i in range(500):
```

and then:

```
for i in range(50_000):
```

Expected Observation

- If each task is too small, overhead dominates.
 - If each task is larger, processes may help more.
 - But if tasks are too large or memory-heavy, speedup may flatten.
-

Additional Experiment 3: A Simple Race Condition Demo

```
import threading
```

```
counter = 0
```

```
def unsafe_increment(n):
```

```
    global counter
```

```
    for _ in range(n):
```

```
        value = counter
```

```
        value += 1
```

```
        counter = value
```

```
def race_condition_demo():
```

```
global counter
```

```
counter = 0
```

```
threads = []
```

```
for _ in range(4):
```

```
    t = threading.Thread(target=unsafe_increment, args=(100_000,))
```

```
    threads.append(t)
```

```
    t.start()
```

```
for t in threads:
```

```
    t.join()
```

```
print("Expected:", 4 * 100_000)
```

```
print("Actual: ", counter)
```

```
race_condition_demo()
```

Then show a safer version:

```
import threading
```

```
counter = 0
```

```
lock = threading.Lock()
```

```
def safe_increment(n):
```

```
    global counter
```

```
    for _ in range(n):
```

```
        with lock:
```

```
        counter += 1

def safe_demo():
    global counter
    counter = 0

    threads = []
    for _ in range(4):
        t = threading.Thread(target=safe_increment, args=(100_000,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    print("Expected:", 4 * 100_000)
    print("Actual: ", counter)

safe_demo()
```

Student Deliverables

By the end of the lab, each student should send a short report including the following 1 and 2. Please send your report to lidongvtemail@gmail.com by email. In your email, use the title “Cluster 4-July 22-your name”.

1. Results Table

Must include:

- worker count

- runtime
- speedup
- efficiency
- correctness check

2. Short Reflection

Suggested prompt:

In 1–2 paragraphs, explain:

1. Which method was fastest for the CPU-bound task?
2. Which method was fastest for the I/O-like task?
3. Where did overhead appear?
4. How does this affect your final project idea?